PASS_FAIL_RULES

PASS / PARTIAL / FAIL / INCONCLUSIVE RULES — EP-AGENT-001

Version: v0.3-DRAFT (2026-10-15) — precedence/anti-rescue audit applied (CHANGELOG v0.3). Frozen at preregistration lock. No result exists. Implemented mechanically in `scripts/analysis/analyze.py` (`classify`).

Notation per `PREREGISTRATION.md` §6. A framework is **fully successful** if all five of its cells are integration-complete, faithful (observed = expected), and linked (P4 sub-conditions met), with no core modification and no material workaround.

Material workaround = any change outside the framework's adapter directory needed to make a cell work (e.g., patching the framework, changing the Request Contract semantics, special-casing a framework in shared code, or manual intervention during a run).

Evaluation order (mechanical)

Step 0 — cell labelling. Each of the 20 cells is labelled **VALID** or **INVALIDATED**. A cell is INVALIDATED only if (a) one of the five preregistered invalidation conditions below is documented for that cell, (b) the cell's shortfall is attributable to it, and (c) the cell shows **no** adapter, normalizer, contract, or verdict defect. A cell with any such defect is VALID (a genuine shortfall) even if an invalidation condition also occurred. A **genuine shortfall** is a VALID cell that is not integration-complete, not faithful, not linked, or needed a material workaround.

1. **FAIL** if any FAIL condition holds on genuine evidence (FAIL condition 3 is met when every framework has ≥1 genuine shortfall). Invalidated cells can never prevent or reverse a FAIL.
2. **PARTIAL (established)** if ≥1 framework is fully successful on five VALID cells **and** ≥1 framework has a genuine shortfall. Invalidated cells elsewhere cannot convert an established PARTIAL into INCONCLUSIVE.
3. **INCONCLUSIVE** if ≥1 cell is INVALIDATED and steps 1–2 did not decide. Any genuine shortfalls are reported as negative findings within the INCONCLUSIVE record. The best attainable classification after a re-run under a new amendment is unchanged by this label; INCONCLUSIVE never upgrades to PASS.
4. **PASS** if all PASS conditions hold (all 20 cells VALID and successful).
5. Otherwise **PARTIAL** if its conditions hold; else **FAIL**.

Worked precedence cases (also encoded in the synthetic self-test): one framework fails one cell while the other three are fully successful → **PARTIAL** (not FAIL). Every framework has a genuine shortfall → **FAIL**. An infrastructure outage coinciding with an adapter defect in the same cell → that cell is a genuine shortfall (no rescue). All shortfalls are infrastructure-only → **INCONCLUSIVE**. P5 = true → **FAIL** regardless of anything else.

PASS — all of:

Endpoint	Threshold
P1 integration completion	20/20
P2 decision fidelity	1.0 (20/20 exact matches)
P3 core semantic invariance	P3_c = 1.0 for each of B1–B5 (and canonical fields + `action_hash` identical per case)

Endpoint	Threshold
P4 ProofRecord linkage	20/20
P5 core modification required	false
Material workarounds	0

PARTIAL

Not PASS and not FAIL, and: ≥1 framework is fully successful, and ≥1 framework fails ≥1 cell or requires a material workaround. The core concept is partially supported. The report names each framework's status per endpoint.

FAIL — any of:

1. P5 = true (any framework required modification of ExecutionProof core code or decision semantics).
2. ≥2 frameworks cannot use the same frozen Request Contract without changing core semantics (field meaning, required set, or canonical hash definition).
3. No framework is fully successful.
4. P3_c < 1.0 for any case due to a divergence attributable to ExecutionProof core behaviour (i.e., identical canonical contracts produced different verdicts).

A FAIL contradicts the primary portability hypothesis and is preserved as a fact. Any later remediation is a separate, new record; it never overwrites the FAIL.

INCONCLUSIVE — only these preregistered invalidation conditions:

1. Upstream framework defect (a bug in the pinned framework release, documented with a reproducible issue, that is outside the adapter).
2. Environment failure (host crash, OS/Python runtime fault).
3. Dependency outage (package index or sandbox dependency unavailable).
4. Corrupted artifacts (fixture, expected-results, or lockfile SHA-256 mismatch versus the frozen manifest).
5. Incomplete execution caused by non-study infrastructure.

INCONCLUSIVE **must not be used to rescue a negative result.** Adapter bugs, normalization errors, contract mismatches, and verdict mismatches are study outcomes, not invalidation conditions.

Framework failure handling

When	Rule
Before execution	A framework may be replaced only by written amendment + CHANGELOG entry + regenerated `MANIFEST.sha256` + new preregistration SHA-256 + gate re-run. The replacement must meet selection criteria C1–C5. Google ADK (`google-adk`) is an **eligible future extension, not included in v1**; its use as a replacement or addition requires a written amendment and does not alter the v1 20-cell matrix.
During execution	No replacement. The failure is preserved per protocol and enters classification as observed (or as INCONCLUSIVE only if an invalidation condition is documented).
After execution	Fixes are new, separately versioned work; the original classification stands.

Secondary endpoints

LOC, setup burden, dependency burden, latency, friction, and configuration surface are reported descriptively and never change the classification.